

Assignment 1: Foundations, Pointers & ADTs

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 12, 2026

Due: two weeks from issue. Late submissions lose 10% per day unless a documented excuse is provided in advance (see the course policy in the syllabus).

Answer every question. Show all working for Numerical and Design questions.

1.1 Conceptual

Q1. A running C program keeps data in two different regions: the **stack** and the **heap**. Explain, using a concrete example, why *both* are needed rather than keeping everything in one region. Give one scenario where a value must live on the heap because it would not survive on the stack, and one scenario where the stack is clearly the right choice.

Q2. Distinguish between a **contract** statement and an **implementation** statement for the bag ADT introduced in class. For each of the following, say whether it is part of the contract or of one implementation, and justify in one sentence:

- (a) “`member(7)` answers yes or no.”
- (b) “The values are stored in an array.”
- (c) “`insert(x)` puts `x` into the bag.”
- (d) “`size()` reports the number of values present.”

1.2 Numerical

Q3. On a 64-bit machine, consider the C declaration

```
struct node {  
    int    data;  
    struct node *next;  
};
```

- (a) State how many bytes `sizeof(struct node)` reports, assuming `sizeof(int) == 4` and a pointer is 8 bytes, with no padding.
- (b) A program executes `struct node *p = malloc(sizeof(struct node));`. Draw (in words or a small diagram) what lives on the stack and what lives on the heap after this line, and what `p` itself holds.

- (c) If `malloc` returns `NULL`, what must the program do before it dereferences `p`, and why?

Q4. A function allocates a heap block and returns a pointer to it:

```
int *make_block(void) {
    int *p = malloc(sizeof(int));
    *p = 80;
    return p;
}
```

- (a) After `make_block` returns, does the value 80 still exist? Which region is it in, and why does it survive the function's return?
- (b) If the caller never calls `free(p)`, what is the name of the mistake, and what are its consequences?
- (c) If the caller calls `free(p)` and then tries to read `*p`, what is the name of the mistake, and what is the safe pattern to avoid it?

1.3 Design

Q5. Write a complete C function `create_node` that:

- (a) Takes an integer `data` as its only parameter.
- (b) Allocates one `struct node` (with fields `data` and `next`) on the heap using `malloc`.
- (c) Checks the result of `malloc` against `NULL` and returns `NULL` on failure (the caller can then decide how to recover).
- (d) Initialises both fields — `data` to the parameter, `next` to `NULL` — and returns the pointer.

State the function's return type and write the body. Your solution must follow the pointer discipline from the lecture: check `malloc`, initialise every field, and use `p->field` form.

Q6. The bag ADT from the lecture has the contract "`insert(x)`, `member(x)`, `size()`."

- (a) Write the ADT's contract as a C header file: a `struct node` typedef and four function prototypes (`bag_insert`, `bag_member`, `bag_size`, `free_bag`) that together honour the contract, using a head pointer and `NULL` for the empty bag.
- (b) State two different implementations that could honour this same contract, and explain in one sentence what the caller gains from coding against the contract rather than against one specific implementation.